

From Game Theory to Machine Learning: Experiments on Reducing Overfitting



1 Introduction

In machine learning, the phenomenon that model does not generalize well from observed data to unseen data is called overfitting (Cogswell et al., 2015). Overfitting occurs when the model is too complex or there are too few samples, which causes the model to wrongly associate the feature with the labels and make incorrect predictions (Simpson et al., 2019). Modern deep neural networks with a large number of parameters are very powerful machine learning systems. However, overfitting is a serious problem in the overparameterized models (Srivastava et al., 2014).

Overfitting is problematic because it restrains the model's ability to generalize well to new data. The reason is that it is difficult for overfitted models to deal with the information in the testing set, which is different from the training set. On the other hand, overfitted models tend to memorize all the information in the training set, including the inevitable noise, and fail to learn the discipline behind the data (Cogswell et al., 2015). There are a variety of traditional ways to reduce overfitting, including dropout, weight decay (L_2 regularization), data augmentation and early stopping.

In this project, we propose a new way to reduce overfitting, which is inspired by the John Harsanyi's game theory (Harsanyi, 1963a). We used the Harsanyi dividend to measure the interaction utility between input variables. In image classification problem, the classification based on massive pixels in the image tends to lead to overfitting. In the extreme case, if a model relies on all pixels of an image to do the classification, then it is terribly overfitted to the training data. Therefore, we define a new loss function to penalize the classification based on massive input variables (specifically, pixels in images).

We conduct the experiments using the new loss

function and find that our method effectively reduces overfitting, but unfortunately, there is no improvement on the validation accuracy. In the future, we will try to improve the validation accuracy while reduce overfitting.

2 Related Work

2.1 Reducing Overfitting

There are a variety of traditional ways to reduce overfitting, including dropout, weight decay (L_2 regularization), data augmentation and early stopping.

Dropout Dropout is a technique that not only prevents overfitting but also provides a way of approximately combining a large number of neural networks with different architectures efficiently (Srivastava et al., 2014). Dropping a unit out means removing it from the neural network together with all of its incoming and outgoing connections temporarily. Figure 1 shows the way dropout works, where the crossed units are dropped. The left figure is a standard neural network with two hidden layers, and the right figure is a thinned network after applying dropout.

Weight Decay Weight decay, also known as L_2 regularization, is a technique that penalizes the model during training based on the magnitude of the weights. Specifically, it adds the penalty of the L_2 norm of all the weights of the model to the loss function. The formula of the loss function is

$$f_{\theta}(x, y) + \lambda ||\theta||^2$$

where θ is the model weights, and λ is a hyperparameter that controls the strength of penalization.

Data Augmentation Data augmentation is a technique that increases the effective size of the training set. There are a variety of ways to do data augmentation. Some popular approaches are flipping,

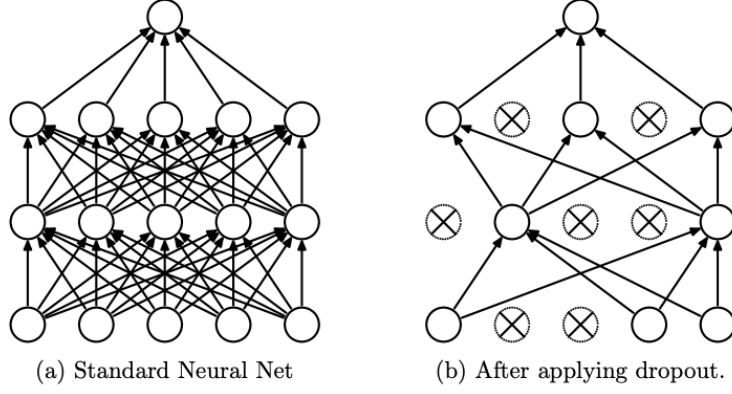


Figure 1: Dropout neural network model

rotation, scaling, cropping and translation. After data augmentation, it is more likely that the given test example has a closely related training example.

Early Stopping Early stopping is a technique that is based on choosing when to stop running an iterative algorithm. Each step of an iterative algorithm will reduce bias but increase variance, and thus early stopping ensures that the variance is not too high (Raskutti et al., 2014). Specifically, in early stopping, the performance of the model is monitored on a validation set and the training is terminated when the performance degrades.

2.2 Game Theory

Morgenstern and Von Neumann (1953) introduced the notion of game with transferable utility, which was defined by a set of players and a characteristic function. The characteristic function was associated to each subset of players a real number that measured what the coalition could do at best in terms of some commodity-money (Dehez, 2019). Shapley introduced the concept of value of a transferable utility game, whose idea was to define what a player might expect from a game (Shapley, 2016). Harsanyi introduced the concept of dividend (Harsanyi, 1963a), whose idea was to associate to each coalition a dividend that could be distributed among its members to define an allocation of the social surplus (Dehez, 2019).

3 Preliminary

3.1 Scalar Output Score

Given a pre-trained model $v(\cdot)$ and an input sample x , where $x \in \mathbb{R}^n$, we use $N = \{1, 2, \dots, n\}$ to denote the set of all input variables, and define $v(N)$ as the scalar output score of the model given

the entire input N . Similarly, we can define $v(S)$ as the scalar output score of the model given only a subset of the input variables $S \subseteq N$. For images, the input variables are pixels. To compute $v(S)$, we preserve the pixels in the set S , and mask the pixels not in S by setting the pixel value to the mean pixel value of the whole dataset.

3.2 Interaction Utility

According to Harsanyi (1963b), we can decompose the output score $v(N)$ into the sum of interaction utilities with respect to all possible subsets of the set of input variables:

$$v(N) = \sum_{S \subseteq N} I(S),$$

where $I(S)$ is recursively defined as

$$I(S) \stackrel{\text{def}}{=} v(S) - v(\emptyset) - \sum_{L \subsetneq S, |L| \geq 1} I(L).$$

Induced by the above equation, we can further obtain a formula to explicitly calculate $I(S)$:

$$I(S) = \sum_{L \subseteq S} (-1)^{|L|-|S|} v(L)$$

Intuitively, $I(S)$ quantifies the interaction utility that is *only* triggered when all the variables in S are present, but not triggered by any smaller subsets of S . We define $|S|$, which is the cardinality of S , as the order of $I(S)$. It is reasonable to assume that the $I(S)$ where $|S|$ is large typically contains the information that lead to overfitting, because this interaction utility is only triggered by the collaboration of massive input variables (specifically, pixels in image classification). In the extreme case, if a

model rely on all pixels of an image to do the classification, then it is terribly overfitted to the training data. Therefore, to reduce overfitting, we want to suppress the utilities of those $I(S)$ with large $|S|$. In the following sections, we call the $I(S)$ with large $|S|$ *high-order*.

4 Method

4.1 High-order Component

We define

$$I_{high}(x, r) = f(x) - \alpha \mathbb{E}_{T \subseteq N, |T|=r} f(x_T)$$

where $I_{high}(x, r)$, $f(x) \in \mathbb{R}^n$, α is the scaling coefficient, x_T denotes the input image where only the pixels in the subset T are present, while other pixels are masked to mean gray value. Mean gray value is the mean of every RGB channel on the whole dataset.

Each dimension of the output vector $f(x)$ is

$$f_i(x) = \sum_{S \subseteq N} I(x_S),$$

while

$$f_i(x_T) = \sum_{S \subseteq T} I(x_S).$$

So $\alpha \mathbb{E}_{T \subseteq N, |T|=r} f(x_T)$ can be a representative of the sum of interaction whose *order* $\leq |T| = r$. The scaling coefficient α helps cancel out the lower-order components in $f_i(x)$. It might have a theoretically optimal value, but in this project, we simply heuristically set α to $1/r$. Therefore, $I_{high}(x, r)$ denotes the high-order components.

4.2 Loss Function

We define the new loss function as

$$L = L_{clf}(f(x), y) - \lambda H(I_{high}(x, r)),$$

where L_{clf} is the original cross entropy loss for classification, H is the entropy, λ is the strength of penalization. To minimize the loss function, the entropy of the output vector given the high-order interactions is maximized. This makes the model unable to use the high-order interactions for classification. In this way, the new loss function can reduce overfitting that is triggered by high-order interactions.

5 Experiment

5.1 Model

We train two models on the image classification task with the original and our novel loss function to observe the effect of reducing overfitting: AlexNet (Krizhevsky, 2014) and VGG16 (Simonyan and Zisserman, 2014). Both of the models are typical convolutional neural network architectures.

5.2 Dataset

The datasets we use are CIFAR-10 (Krizhevsky et al., 2009) and tiny ImageNet (Chrabaszcz et al., 2017). The CIFAR-10 dataset contains 60000 32x32 colour images in 10 classes, with 6000 images per class. To magnify overfitting, we use 20% of the training set. Tiny ImageNet is a subset of the ImageNet dataset (Deng et al., 2009) in the famous ImageNet Large Scale Visual Recognition Challenge. The dataset contains 100,000 images of 200 classes (500 for each class) downsized to 64x64 colored images. Each class has 500 training images, 50 validation images, and 50 test images. We resize the images to 224×224 . Due to the limitation of computing resources, we choose 50 categories. In addition, we apply data augmentation to the data, including random horizontal flip and random crop.

5.3 Training Protocol

We use the SGD optimizer with *momentum* = 0.9, *weight decay* = 0. For the learning rate, we initialize it with 0.01, make it decay by 1 in the logspace (i.e. $0.01 \rightarrow 0.001$), and train for 200 epochs.

6 Results

6.1 Comparison of Loss Curves

We first compare the loss curves of different models on different datasets using the normal training protocol and our proposed method. As Figure 6.1 shows, we find that our method can effectively suppress the validation loss, especially in the later training stage, where overfitting typically takes place. This shows that our method can successfully reduce the overfitting.

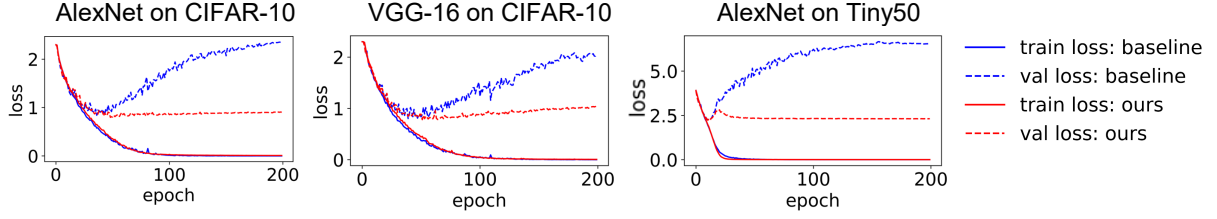


Figure 2: Comparison of loss curves

	AlexNet on CIFAR-10	VGG-16 on CIFAR-10	AlexNet on Tiny50
Normal Training	77.8	78.49	46.48
Our method	75.28	77.19	46.52

Table 1: Comparison of validation accuracy

6.2 Comparison of Accuracy

In Table 1, we compare the validation accuracy of different models on different datasets using the normal training protocol and our proposed method. We find that in general, there is no improvement to the validation accuracy. For the CIFAR-10 dataset which contains small images, the validation accuracy of the models using our training method is even worse than the ones trained normally. However, we witness an obvious drop of validation loss in Section 6.1. Normally, the validation accuracy should increase if the validation loss drops. It is strange to see the discrepancy between the validation accuracy and the validation accuracy. We will provide some discussion on this phenomenon in the next section.

7 Discussion

The discussion is divided into two parts.

Why does not the validation accuracy decrease when the validation loss increases? As shown in Figure 7, for the normally trained model, the validation loss increases significantly in the later stage of training, but the validation accuracy does not go down. One possible explanation for this phenomenon is that the increase of validation loss is because the model is more confident in both the correct and incorrect prediction. We first define the cross entropy loss $CE(x, y) = -\log(p(y|x)) =: h(p)$ as a function of p , where p is the posterior probability of the ground truth category given the sample x . For the correctly classified samples, the model raises its probability p of the ground truth category from 0.9 to 0.99. In this case, the cross entropy loss $h(p)$ will decrease from 0.11 to 0.01, which is closer to 0, but has little change in

value. However, for the misclassified samples, if the model decreases its probability p of the ground truth category from 0.1 to 0.01, then the cross entropy loss $h(p)$ will increase from 2.30 to 4.61. That is to say, if the model becomes more confident in its wrong predictions, the cross entropy loss will significantly increase, which might lead to the phenomenon that the validation loss increases in the later stage of training. In other words, the validation loss itself might not be a good measure of overfitting.

Using our method, the validation loss does not rise in the later stage of training, but the final validation accuracy does not improve, why is that? As shown in Figure 7 (left), our method effectively suppresses the validation loss. However, the validation accuracy of the model trained by our method is on par with the normally trained model. The reason for this phenomenon is very related to the previous part of discussion. Reducing the validation loss does not necessarily lead to higher validation accuracy. It is also likely that we just prevent the model from becoming more confident on the misclassified samples.

8 Future Plans

For future work, we are going to improve this project in the following aspects.

- We will consider using label corruption to explicitly trigger overfitting. The images with wrong labels can be seen as overfitted samples when the training error approaches zero. In

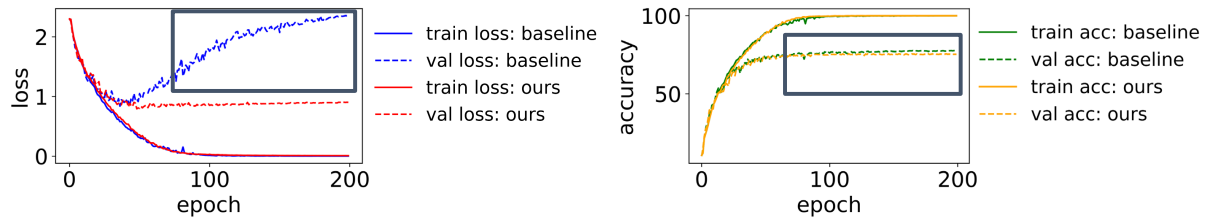


Figure 3: Comparison of validation loss and validation accuracy on normally trained models

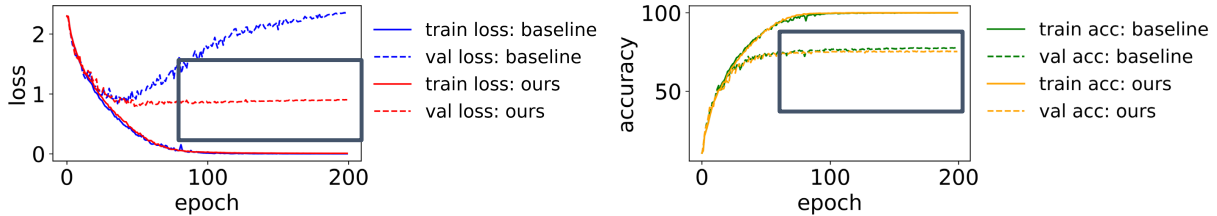


Figure 4: Comparison of validation loss and validation accuracy on models trained using our method

this way, we can also control the extent of overfitting by adjusting the corruption ratio.

- We will try to directly penalize the term $I_{high}(x, r)$ instead of using entropy to penalize it.
- We will tune the hyperparameters such as α and λ in a more fine-grained manner.
- We will do experiments in tabular data besides image data. Tabular data is well aligned in each input dimension, so it is simple to analyze.

References

- Patryk Chrabaszcz, Ilya Loshchilov, and Frank Hutter. 2017. A downsampled variant of imagenet as an alternative to the cifar datasets. *arXiv preprint arXiv:1707.08819*.
- Michael Cogswell, Faruk Ahmed, Ross Girshick, Larry Zitnick, and Dhruv Batra. 2015. Reducing overfitting in deep networks by decorrelating representations. *arXiv preprint arXiv:1511.06068*.
- Pierre Dehez. 2019. *On Harsanyi Dividends and Asymmetric Values*, pages 523–558.
- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. *Imagenet: A large-scale hierarchical image database*. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255.
- John C Harsanyi. 1963a. A simplified bargaining model for the n-person cooperative game. *International Economic Review*, 4(2):194–220.
- John C. Harsanyi. 1963b. *A simplified bargaining model for the n-person cooperative game*. *International Economic Review*, 4(2):194–220.
- Alex Krizhevsky. 2014. *One weird trick for parallelizing convolutional neural networks*. *CoRR*, abs/1404.5997.
- Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images.
- Oskar Morgenstern and John Von Neumann. 1953. *Theory of games and economic behavior*. Princeton university press.
- Garvesh Raskutti, Martin J Wainwright, and Bin Yu. 2014. Early stopping and non-parametric regression: an optimal data-dependent stopping rule. *The Journal of Machine Learning Research*, 15(1):335–366.
- Lloyd S Shapley. 2016. *17. A value for n-person games*. Princeton University Press.
- Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.
- Becks Simpson, Francis Dutil, Yoshua Bengio, and Joseph Paul Cohen. 2019. Gradmask: Reduce overfitting by regularizing saliency. *arXiv preprint arXiv:1904.07478*.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958.